

深圳大学实验报告

课程名称 机器学习

项目名称 深度神经网络

学 院 计算机与软件学院

专 业 人工智能卓越班

指导教师 魏军

报 告 人 邓瑞霖 学号 2024150040

实验时间 2026年5月28日 - 2026年6月12日

提交时间 2026年5月28日星期四

教务处制

一、实验目的

- 掌握多层感知机 (Multi-Layer Perceptron, MLP) 的基本结构，包括输入层、隐藏层、输出层及其连接方式。
- 理解激活函数 (如 ReLU, Sigmoid) 的作用，以及梯度下降算法如何更新网络权重。
- 学习如何在分类任务中使用交叉熵损失 (Cross-Entropy Loss)。
- 熟悉使用 `scikit-learn` 或简易深度学习框架搭建神经网络并进行训练的过程。

二、实验内容

- 参考“实验教程.docx”并完成给出的实验内容、练习题与思考题，将操作过程和操作结果截图记录下来，写在下面的实验记录中。

三、注意事项

- 独立完成实验内容和报告，禁止抄袭，若发现抄袭，**抄袭以及被抄袭的报告均会被判为零分**
- 实验报告的总篇幅不少于 **7 页**
- 将实验报告和可视化的结果图压缩到一个 zip 文件中
- 命名规则，压缩包命名“**学号+姓名+实验 6.zip**”，压缩包内每个文档命名如下，方便老师知道每个结果图分别对应哪一部分：
“学号+姓名+实验报告.docx”，
“学号+姓名+【题目简写】.png”，
...
“学号+姓名+【题目简写】.png”
- 本次实验共 **4 个学时**，分两次实验课完成，实验报告提交截止日期：**2026 年 06 月 12 日晚 上 23:59**

四、实验记录

(请将实验操作过程和操作结果记录下来，截图展示并加文字描述)

4.1 数据准备

用 Scikit-Learn 自带官方数据集 (sklearn.datasets.load_iris), 加载 Iris 数据集。并进行数据标准化。将数据划分为训练集 (80%) 和测试集 (20%)

Iris 数据描述: 包含 150 个样本, 4 个特征 (花萼长度、花萼宽度、花瓣长度、花瓣宽度)。

分类目标: 将鸢尾花划分为三个品种 (Setosa, Versicolour, Virginica)

```
def prepare_iris_data(test_size: float = 0.2, random_state: int = 42):  
  
    iris = load_iris()  
    X = iris.data  
    y = iris.target  
  
    X_train, X_test, y_train, y_test = train_test_split(  
        X, y, test_size=test_size, random_state=random_state, stratify=y  
    )  
  
    scaler = StandardScaler()  
    X_train = scaler.fit_transform(X_train)  
    X_test = scaler.transform(X_test)  
  
    return X_train, X_test, y_train, y_test, scaler
```

最终得到训练集的维度: (120, 4)

最终得到测试集的维度: (30, 4)

4.2 构建多层感知机 (MLP)

使用 sklearn.neural_network.MLPClassifier 创建模型。

初始设置: 1 个隐藏层, 包含 10 个神经元; 激活函数选择 relu; 优化器选择 adam

```
model = MLPClassifier(  
    hidden_layer_sizes=(10,),  
    activation="relu",  
    solver="adam",  
    random_state=42,  
    max_iter=500,  
)  
model.fit(X_train, y_train)  
loss_curve = model.loss_curve_  
  
plt.figure(figsize=(6, 4))  
plt.plot(range(1, len(loss_curve) + 1), loss_curve)  
plt.xlabel("Epoch")  
plt.ylabel("Loss")  
plt.title("MLP Training Loss Curve")  
plt.grid(True, linestyle="--", alpha=0.5)  
plt.tight_layout()  
plt.savefig("./output/2024150040邓瑞霖MLP_loss.png")  
plt.show()  
y_pred = model.predict(X_test)  
acc = accuracy_score(y_test, y_pred)  
cm = confusion_matrix(y_test, y_pred)  
print("最终的loss:", loss_curve[-1])  
print("测试集准确率:", f"{acc:.4f}")  
print("混淆矩阵:\n", cm)
```

绘制出训练时的 loss 变换曲线（如图 1）：

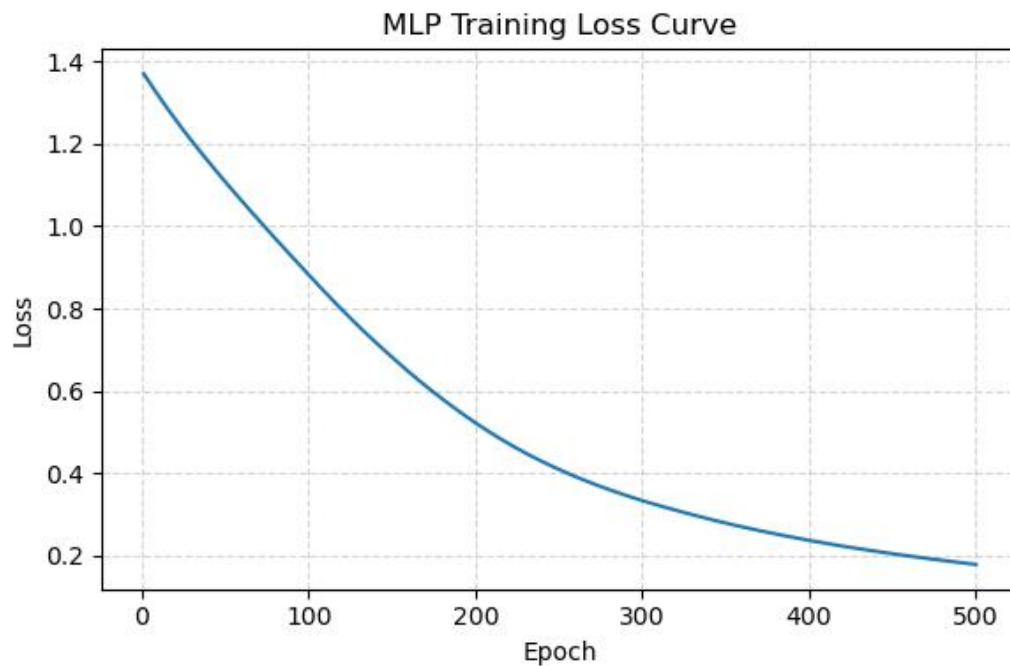


图 1 用 Iris 数据集训练 MLP 时 loss 曲线图

可以看出，在训练的时候 loss 下降先急后缓，故训练有效果。

最终的 loss 值为：0.1787489

在测试集上的准确率为：0.9667

输出的混淆矩阵为：

混淆矩阵：

$\begin{bmatrix} 10 & 0 & 0 \end{bmatrix}$

$\begin{bmatrix} 0 & 9 & 1 \end{bmatrix}$

$\begin{bmatrix} 0 & 0 & 10 \end{bmatrix}$

4.3 网络结构对性能的影响

4.3.1 设置不同的隐藏层神经元的数量

保持学习率、激活函数等其他参数不变。分别设置隐藏层神经元数量为 [2, 5, 20, 100]。

```

hidden_sizes = [2, 5, 20, 100]
results = []

for size in hidden_sizes:
    model = MLPClassifier(
        hidden_layer_sizes=(size,),
        activation="relu",
        solver="adam",
        random_state=42,
        max_iter=3000,
    )
    model.fit(X_train, y_train)
    loss_curve = model.loss_curve_

    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    results.append({
        "size": size,
        "n_iter": model.n_iter_,
        "acc": acc,
        "final_loss": loss_curve[-1],
        "loss_curve": loss_curve,
    })

```

在每种模型下的迭代次数、测试准确率以及最终的 loss

对比实验结果：

隐藏层神经元=2，迭代次数(n_iter_)=2713，测试准确率=0.9667，最终 loss=0.288381

隐藏层神经元=5，迭代次数(n_iter_)=1161，测试准确率=0.9667，最终 loss=0.108013

隐藏层神经元=20，迭代次数(n_iter_)=737，测试准确率=0.9667，最终 loss=0.069241

隐藏层神经元=100，迭代次数(n_iter_)=497，测试准确率=0.9667，最终 loss=0.050764

将每种模型的训练 loss 变化曲线进行对比（如[图 2](#)）：

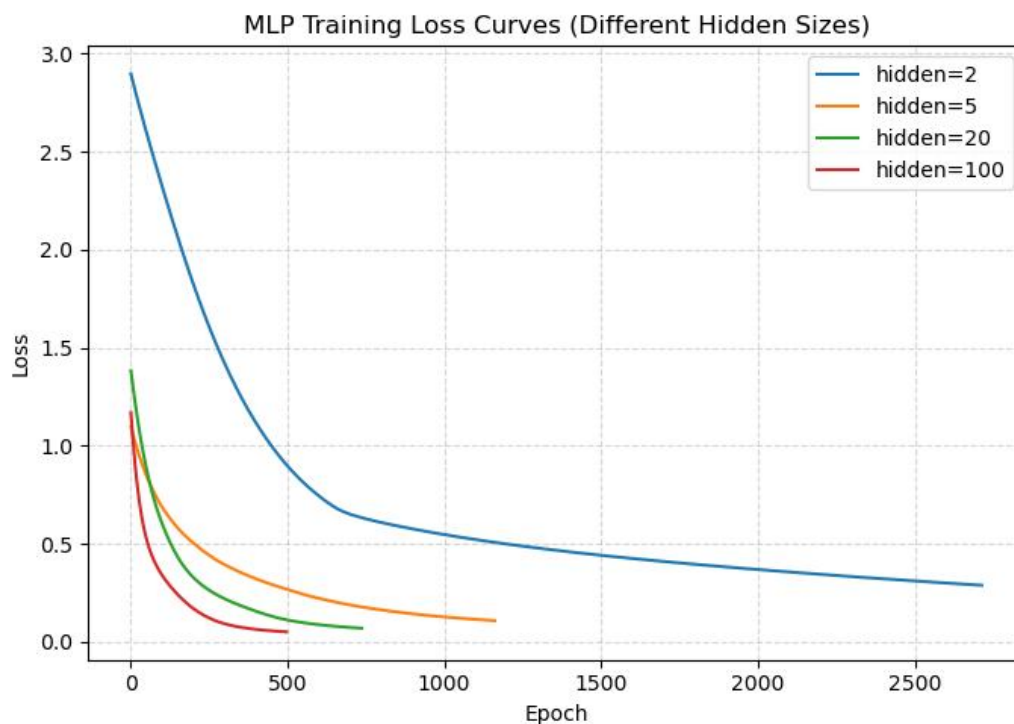


图 2 不同神经元的 loss 变化曲线

神经元过少（如 2）可能导致欠拟合，收敛速度与精度受限，最终的 loss 值为 0.288381，相比之下 loss 值过高，但训练时曲线仍可以平滑下降；神经元过多（如 100）虽然表达能力更强，但计算开销更大，迭代次数过多，也可能带来过拟合风险

4.3.2 设置不同的激活函数

分别设置三个不同的激活函数：relu, tanh, logistic

对应的公式分别为：

$$ReLU(x) = \begin{cases} 1, & x > 0 \\ 0, & x \leq 0 \end{cases}$$

$$Tanh(x) = \frac{1 - e^{-2x}}{1 + e^{-2x}}$$

$$Logistic = \frac{1}{1 + e^{-x}}$$

代码：

```

activate = ["relu", "tanh", "logistic"]
results = []

for size in activate:
    model = MLPClassifier(
        hidden_layer_sizes=(10,),
        activation=size,
        solver="adam",
        random_state=42,
        max_iter=3000,
    )
    model.fit(X_train, y_train)
    loss_curve = model.loss_curve_

```

在每种模型下的迭代次数、测试准确率以及最终的 loss:

激活函数=relu, 迭代次数(n_iter_)=948, 测试准确率=0.9667, 最终 loss=0.077651
 激活函数=tanh, 迭代次数(n_iter_)=1179, 测试准确率=0.9667, 最终 loss=0.089957
 激活函数=logistic, 迭代次数(n_iter_)=1671, 测试准确率=0.9667, 最终 loss=0.112103

将每种模型的训练 loss 变化曲线进行对比 (如图 3) :

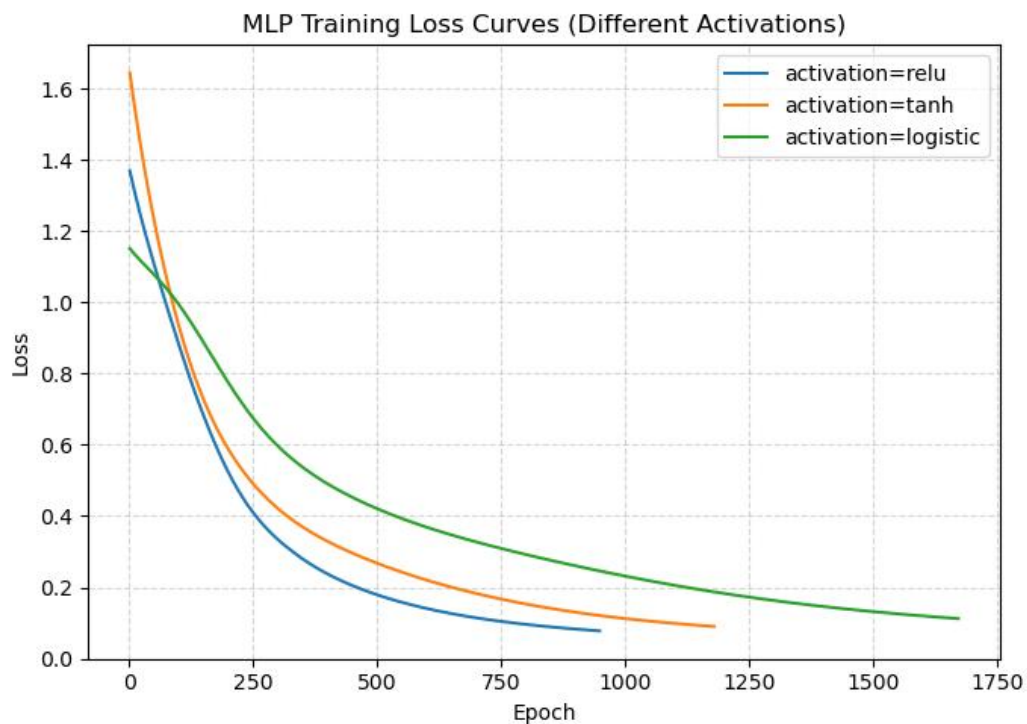


图 3 不同激活函数下训练时 loss 变化图

从曲线形态可以看出三种激活函数在收敛速度和最终损失上存在明显差异:

ReLU (蓝色): 下降最快, 前期损失下降最陡, 较早达到较低的 loss, 说明收敛速度最快、优化更顺畅。

tanh (橙色)：表现居中，下降速度略慢于 **ReLU**，但仍能较稳定地收敛。

logistic (绿色)：下降最慢，**loss** 长时间维持在较高水平，收敛速度最慢，最终 **loss** 也相对更高

4.4 思考题

1. 激活函数的必要性：如果去掉隐藏层的激活函数（即使用线性映射），这个多层神经网络在逻辑上等价于什么模型？

答：若去掉隐藏层的激活函数（全是线性映射），多层网络的线性变换可合并为一次线性变换，因此在逻辑上等价于单层线性模型。

在分类任务中，这相当于线性分类器（例如 **softmax** 回归/多类逻辑回归；二分类时等价于逻辑回归或线性判别）。

2. 局部最优与随机性：由于神经网络的权重是随机初始化的，多次运行同样的训练代码，得到的结果是否完全一致？如何通过 **random_state** 参数保证实验的可重复性？

答：一般不会完全一致。

神经网络训练中存在多处随机因素，哪怕代码不变，每次运行也可能出现不同的收敛轨迹、不同的损失曲线、以及略微不同的测试准确率。常见随机来源包括：

- 权重初始化随机：**MLP** 的权重通常用随机分布初始化，不同初始化会导致参数起点不同，从而进入不同的优化路径。

- 数据打乱 (**shuffle**)：训练样本的顺序会影响梯度更新方向，尤其是小数据集时差异更明显。

- 优化过程的随机性：像 **Adam** 这类优化器在数值上对梯度方向比较敏感，微小差异会累积成不同结果。

神经网络训练存在随机初始化与数据打乱等随机因素。为确保实验结果可复现，本实验在 **train_test_split** 和 **MLPClassifier** 中设置相同的 **random_state**（如 42），从而固定数据划分与参数初始化，保证多次运行得到基本一致的损失曲线与准确率

3. 过拟合迹象：如果在训练集上准确率接近 **100%**，但在测试集上表现很差，这说明了什么？你会尝试调整哪些超参数来缓解这个问题？

答：在训练集上准确率接近 **100%**，但在测试集上表现很差，这说明了在训练集上学到了“记忆式”的细节甚至噪声，但泛化能力差，导致测试集表现明显下降。缓解这个问题的方法：添加正则化项，减小隐藏层神经元数量，降低模型复杂度或开启早停等方法提升模型的泛化能力

五、实验总结

本实验以 Iris 数据集为对象，构建多层感知机 (MLP) 分类模型，在统一优化器 (Adam)、学习率和训练轮次的条件下，系统比较了隐藏层神经元数量与激活函数选择对模型收敛速度和分类性能的影响。

首先，在隐藏层规模对比中，神经元数量过少 (如 2 或 5 个) 时，模型表达能力受限，损失下降缓慢，测试集准确率偏低，体现出明显的欠拟合倾向。随着隐藏层神经元数量增加 (如 20 或 100)，模型拟合能力增强，训练损失下降更充分，测试集准确率有所提升，但同时训练迭代次数增加、计算开销变大，且过多神经元可能带来过拟合风险，说明模型复杂度需在精度与成本之间取得平衡。

其次，在激活函数对比实验中，ReLU 的收敛速度最快，损失曲线下落最稳定，通常能在更少的迭代内达到较低损失；tanh 表现居中；logistic 由于梯度饱和问题，下降最慢，最终 loss 较高。该结果说明激活函数对梯度传播与训练效率有显著影响，ReLU 更适合作为该任务的默认选择。

综合来看，本实验验证了 MLP 模型性能对超参数配置高度敏感：隐藏层规模控制了模型复杂度与泛化能力，激活函数影响优化路径与收敛效率。因此，在实际应用中应结合任务规模、数据量与泛化需求进行合理调参，从而在准确率、收敛速度与计算成本之间实现折中。

指导教师批阅意见:

成绩评定:

指导教师签字:

年 月 日

备注: